COMPUTER NETWORKS

ASSIGNMENT 2 – CO2

CO2 Coverage: Configure IP addressing schemes, subnetting and basic routing techniques using a simulation tool, for efficient network design and topology – packet switching, IPv4 addressing, subnetting, distance-vector and link-state routing, multicast, ICMP, IPv6, software-defined networking, network virtualisation and OpenFlow.

Nature of Assignment: Individual
Duration: 1 Week
Total Marks: 50
Simulation Tool: Cisco Packet Tracer (or GNS3)
Submission Date: 04/11/2026

Bloom's Levels Assessed: K5 – Evaluate (every task asks you to calculate or analyse first, and then judge, decide or justify)

Mandatory Execution and GitHub Submission

- **Every student must individually execute every task in this assignment**, including all "Complete the Code" programs.
- Create a GitHub repository named **CSA07-<RegisterNumber>-Assignment2** and share it with the course faculty (public, or private with the faculty added as a collaborator).
- Keep one folder for each task (for example: Task01, Task02 … or Q1, Q2 …), containing the Python programs, Packet Tracer (.pkt) files, device configurations, Wireshark captures, command outputs and screenshots.
- Include a README.md with your name, register number, the operating system, Python version and Packet Tracer (or NS-3) version used, the steps to run each program or open each simulation, and a short note of the result of each task.
- Commit your work progressively – at least one commit for each task. The commit history will be checked; a single upload of all files on the last day will not be accepted.
- Submit the GitHub link below and on the cover page of your report. **An assignment submitted without a working GitHub link will not be evaluated.**

GitHub Repository Link: https://github.com/mithrad522-art/CSA0707-COMPUTER-NETWORKS/tree/main/ASSIGNMENTS

Student Name: MITHRA DEVI S

Register number: 192524203

How to Answer the Additional Questions

Complete the Code: Copy the given program or device configuration into your repository, replace every TODO with working code or commands without changing the rest, run it (Python 3 or Cisco Packet Tracer) and compare your output with the expected output or result.

Alternative Concept Analysis: Describe how the solution would change if the alternative concept were used, and evaluate both solutions using network measures such as throughput, delay, jitter, packet loss, overhead, reliability, security and cost, supported by your own calculations, simulations or measurements wherever possible.

Q1 – Spot the Invalid Addresses

Bloom's Level: K5 – Evaluate

Problem Context – Address Clashes in a Hospital Group

A hospital group's IT team has collected the addresses configured on some devices. Several devices cannot communicate, and one clinic says it has "run out of addresses".

Admission PC	172.16.45.200 /20	gateway 172.16.32.1
Lab printer	10.0.15.255 /21	gateway 10.0.8.1
X-ray server	192.168.4.0 /23	gateway 192.168.4.1
Clinic router	203.0.113.10 /29	(public, from the internet provider)

Your Challenge

For each device, find the network address, broadcast address and usable host range, and decide whether the address is a valid host address. Two of them contain a trap.

Inquiry

The clinic's 18 devices share the one public /29 address block through NAT. Show how the NAT table lets two PCs both use source port 50000 at the same time. Then explain why packets of the same file can arrive in a different order (datagram switching), and what would be different with virtual circuits.

Alternative Concept Analysis

Suppose the clinic were given a public /27 instead of using NAT behind a /29. Count the public addresses needed for 18 devices, state the cost and exposure to the internet, and identify which application problems NAT would no longer cause.

Evidence Expected

Correct network, broadcast and host ranges, the two traps identified with reasons, a NAT table example and a reasoned comparison of datagram and virtual-circuit switching.

Solution:

1. IPv4 address analysis

Device	Address/prefix	Mask	Network	Broadcast	Usable host range	Host valid?
Admission PC	172.16.45.200/20	255.255.240.0	172.16.32.0	172.16.47.255	172.16.32.1–172.16.47.254	Yes
Lab printer	10.0.15.255/21	255.255.248.0	10.0.8.0	10.0.15.255	10.0.8.1–10.0.15.254	No: broadcast address
X-ray server	192.168.4.0/23	255.255.254.0	192.168.4.0	192.168.5.255	192.168.4.1–192.168.5.254	No: network address
Clinic router	203.0.113.10/29	255.255.255.248	203.0.113.8	203.0.113.15	203.0.113.9–203.0.113.14	Host-valid mathematically

Gateway check: 172.16.32.1 is inside the Admission PC subnet; 10.0.8.1 is inside the printer subnet; and 192.168.4.1 is inside the X-ray subnet. The latter two device addresses are invalid even though their gateways are in the correct subnets.

Real-world caveat: 203.0.113.0/24 is TEST-NET-3, reserved for documentation and not globally routable on the public Internet. Thus 203.0.113.10 is a valid host address within the example /29 mathematically, but it should not be used as a real ISP-assigned public address.

2. NAT/PAT table

With Port Address Translation (PAT), multiple private hosts can use the same public IP address and the same private source port because PAT assigns different translated public source ports.

Inside local (private endpoint)	Inside port	Public IP	Translated public port	Remote endpoint
192.168.1.10	50000	198.51.100.10	40001	198.51.100.80:443
192.168.1.11	50000	198.51.100.10	40002	198.51.100.80:443

The router tracks each translation and uses the address/port tuple to return replies to the correct private PC. The public IP above is illustrative; replace it with the clinic's real ISP-assigned address.

3. Datagram switching vs virtual circuits

Datagram packet switching	**Virtual-circuit switching**
No path setup is required; each packet is routed independently.	A logical path/state is established before data transfer.
Packets may use different paths and arrive out of order.	Packets generally follow the same path and are more likely to arrive in order.
Routers make forwarding decisions per packet using the destination address.	Forwarding uses the established virtual-circuit identifier/state.
A failure may cause packets to take another route, with possible reordering.	A path failure may disrupt the circuit until it is repaired or re-established.

Packets belonging to one file can arrive out of order with datagram switching because different packets can experience different path lengths, congestion, queuing, and retransmission delays. The receiving protocol/application reorders data as needed.

4. Alternative: public /27 instead of NAT behind /29

A /29 contains 8 total addresses and 6 conventional usable host addresses.

A /27 contains 32 total addresses and 30 conventional usable host addresses.

Eighteen devices each needing a distinct public address use 18 addresses; if the router/firewall interface also needs an address, plan for at least 19 usable addresses. A /27 provides 30, so it is sufficient under this assumption.

The /27 generally costs more because it uses a larger public allocation; the actual price depends on the ISP and cannot be calculated from the assignment data.

Direct public addressing can simplify some inbound connections, peer-to-peer applications, and protocols that embed addresses or have NAT traversal issues. NAT-related port mapping and some end-to-end connectivity issues are reduced.

Public addresses do not automatically make devices unsafe, but they can be directly reachable. Apply firewalls, access control, patching, and least-privilege rules. NAT is not a substitute for a firewall.

Q2 – Subnet a Campus That Keeps Growing

Bloom's Level: K5 – Evaluate

Problem Context – Engineering College Campus

A college has 192.168.10.0/24 for laboratories (100 hosts), staff (50 hosts), the library (25 hosts) and a point-to-point WAN link (2 hosts).

Your Challenge

Design the subnets twice – with equal-size subnets and with VLSM – giving network, prefix, mask, first and last host and broadcast for each. Then answer the trap: six months later the college adds a hostel network of 60 hosts. Can it be fitted into the remaining space of your VLSM plan without renumbering? If not, what is the smallest change that makes it fit?

Route Summarisation

The city campus uses 172.16.8.0/24, 172.16.9.0/24, 172.16.10.0/24 and 172.16.11.0/24. Find the single summary route, and show what goes wrong if 172.16.12.0/24 is added later and someone summarises it as 172.16.8.0/21.

Complete the Code

Complete TODO 1 below, and run the program with Python 3.

```
"""Variable-length subnetting (VLSM) of 192.168.10.0/24 for a college campus."""
import math


def to_int(ip):
    a, b, c, d = map(int, ip.split("."))
    return (a << 24) | (b << 16) | (c << 8) | d


def to_ip(n):
    return ".".join(str((n >> s) & 255) for s in (24, 16, 8, 0))


def vlsm(base, needs):
addr = to_int(base)
    for name, hosts in sorted(needs, key=lambda x: -x[1]):
        # TODO 1: Find the smallest block that holds hosts + 2 addresses (a power of
        #   two), its prefix length 32 - log2(block), and print the network, prefix,
        #   first and last usable host and broadcast address. Then move addr to the next
        #   block
pass  # replace this line with your code
    return to_ip(addr)


needs = [("Labs", 100), ("Staff", 50), ("Library", 25), ("WAN link", 2)]
print("next free address:", vlsm("192.168.10.0", needs))
```

Expected output:

```
Labs        192.168.10.0/25  hosts 192.168.10.1 - 192.168.10.126  bcast 192.168.10.127
Staff       192.168.10.128/26  hosts 192.168.10.129 - 192.168.10.190  bcast 192.168.10.191
Library     192.168.10.192/27  hosts 192.168.10.193 - 192.168.10.222  bcast 192.168.10.223
WAN link    192.168.10.224/30   hosts 192.168.10.225 - 192.168.10.226   bcast
192.168.10.227
next free address: 192.168.10.228
```

Check every subnet with Python's ipaddress module, then add the 60-host hostel network and see what happens.

Alternative Concept Analysis

Redo the college plan starting from the smallest subnet instead of the largest. Show where the allocation goes wrong (overlapping or misaligned blocks) and explain why VLSM must assign the largest subnets first.

Evidence Expected

Complete subnet tables for both methods, a correct answer to the hostel trap, a correct summary route and an explained summarisation error, verified in Packet Tracer.

Solution:

1. Equal-size subnetting

The supplied /24 has 256 total addresses. If divided into four equal subnets, each subnet is /26 (64 total, 62 usable). This is a valid equal-size split, but it does NOT satisfy all four host requirements because Labs needs 100 hosts. If each equal subnet were large enough for 100 hosts, each would need /25 (126 usable), and four /25s would require 512 addresses, not the available 256. Therefore, no four-equal-subnet plan can satisfy all requirements inside this /24.

Department	Network/prefix	Mask	First host	Last host	Broadcast	Capacity result
Labs	192.168.10.0/26	255.255.255.192	192.168.10.1	192.168.10.62	192.168.10.63	FAIL: needs 100; only 62 usable
Staff	192.168.10.64/26	255.255.255.192	192.168.10.65	192.168.10.126	192.168.10.127	Pass: 50 needed
Library	192.168.10.128/26	255.255.255.192	192.168.10.129	192.168.10.190	192.168.10.191	Pass: 25 needed
WAN	192.168.10.192/26	255.255.255.192	192.168.10.193	192.168.10.254	192.168.10.255	Oversized: 2 needed

2. VLSM plan

Allocate the largest subnet first. Labs needs /25 (126 usable), Staff /26 (62 usable), Library /27 (30 usable), and the point-to-point WAN /30 (2 usable).

Department	Network/prefix	Mask	First host	Last host	Broadcast	Usable hosts
Labs	192.168.10.0/25	255.255.255.128	192.168.10.1	192.168.10.126	192.168.10.127	126
Staff	192.168.10.128/26	255.255.255.192	192.168.10.129	192.168.10.190	192.168.10.191	62
Library	192.168.10.192/27	255.255.255.224	192.168.10.193	192.168.10.222	192.168.10.223	30
WAN link	192.168.10.224/30	255.255.255.252	192.168.10.225	192.168.10.226	192.168.10.227	2

The next sequential free address is 192.168.10.228, but it is not itself the start of a /26 block.

3. Hostel trap and smallest change

A 60-host hostel needs a /26 (64 total, 62 usable). The remaining addresses in the original plan are 192.168.10.228–192.168.10.255, only 28 addresses, and they are not a correctly aligned /26 block. More importantly, the total allocated block sizes would be 128 + 64 + 32 + 4 + 64 = 292 addresses, which exceeds the 256 addresses in a /24. Renumbering alone cannot fit all five subnet blocks inside the same /24.

Smallest practical change: obtain an additional address block large enough for the hostel (at least one aligned /26) if a separate block is permitted. If the design must use one contiguous CIDR pool, expand/reallocate the address pool to a /23 and redesign the subnets. Q7 shows a /23 redesign that includes the hostel.

4. Route summarisation

The four networks 172.16.8.0/24 through 172.16.11.0/24 are contiguous and align to the summary route 172.16.8.0/22. This summary covers 172.16.8.0–172.16.11.255.

If 172.16.12.0/24 is added, the five /24 networks cannot be represented exactly by one CIDR summary. Summarising them as 172.16.8.0/21 covers 172.16.8.0–172.16.15.255, including 172.16.13.0/24 through 172.16.15.0/24, which may be unallocated. Traffic for those extra networks could be sent toward the summarising router and be dropped (a black hole). A safer exact advertisement is 172.16.8.0/22 plus 172.16.12.0/24, unless the wider /21 is intentionally used with suitable more-specific routes or discard handling.

5. Completed VLSM Python program

```
"""Variable-length subnetting (VLSM) of 192.168.10.0/24 for a college campus."""
import math
def to_int(ip):
    a, b, c, d = map(int, ip.split("."))
    return (a << 24) | (b << 16) | (c << 8) | d
def to_ip(n):
    return ".".join(str((n >> s) & 255) for s in (24, 16, 8, 0))
def vlsm(base, needs):
    addr = to_int(base)
    for name, hosts in sorted(needs, key=lambda x: -x[1]):
```

```
        block = 1 << math.ceil(math.log2(hosts + 2))
        prefix = 32 - int(math.log2(block))
        network = addr
        broadcast = network + block - 1
first = network + 1
last = broadcast – 1
        print(
            f"{name:<12} {to_ip(network)}/{prefix} "
            f"hosts {to_ip(first)} - {to_ip(last)} "
            f"bcast {to_ip(broadcast)}"
        )
        addr = broadcast + 1
    return to_ip(addr)
needs = [("Labs", 100), ("Staff", 50), ("Library", 25), ("WAN link", 2)]
print("next free address:", vlsm("192.168.10.0", needs))
```

output:

```
Output
Labs         192.168.10.0/25 hosts 192.168.10.1 - 192.168.10.126 bcast 192.168.10.127
Staff        192.168.10.128/26 hosts 192.168.10.129 - 192.168.10.190 bcast
192.168.10.191
Library      192.168.10.192/27 hosts 192.168.10.193 - 192.168.10.222 bcast
192.168.10.223
WAN link     192.168.10.224/30 hosts 192.168.10.225 - 192.168.10.226 bcast
192.168.10.227
next free address: 192.168.10.228
```

Verification snippet using Python's ipaddress module:

```
import ipaddress
 for cidr in [
   "192.168.10.0/25", "192.168.10.128/26",
    "192.168.10.192/27", "192.168.10.224/30"
]:
    net = ipaddress.ip_network(cidr)
    print(cidr, "network =", net.network_address,
          "broadcast =", net.broadcast_address,
          "first =", net[1], "last =", net[-2],
```

```
    "usable =", net.num_addresses - 2)
```

6. Why largest-first VLSM is safer

A VLSM block must start at an address aligned to its block size. Allocating the largest blocks first reserves the biggest aligned ranges before smaller blocks fragment the remaining space. Starting with /30 and /27 blocks can place smaller allocations at the beginning of the pool and leave no aligned /25 or /26 range for the larger departments, even if the raw count of free addresses looks sufficient.

Q3 – Which Way Will the Packet Go?

Bloom's Level: K5 – Evaluate

Problem Context – Two Campuses and a Backup Link

The main-campus router has this routing table:

```
S    10.0.0.0/8        via 192.0.2.1
S    10.1.0.0/16       via 192.0.2.5
S    10.1.2.0/24       via 192.0.2.9
S    10.1.2.128/25     via 192.0.2.13
S*   0.0.0.0/0         via 198.51.100.1
```

Your Challenge

Determine the next hop for packets to 10.1.2.200, 10.1.2.100, 10.1.9.9, 10.7.7.7 and 8.8.8.8, and state the rule you used. Then build the two-campus network in Packet Tracer with static routes, add a backup link with a floating static route (administrative distance 200), and show that traffic moves to the backup when the main link is shut down.

Complete the Code

Complete TODO 1 and TODO 2 in the router configuration below, and enter it on R1 in Cisco Packet Tracer. Configure R2 in the same way with the reverse route.

```
! Cisco Packet Tracer: static routing between the Main campus (R1) and City campus (R2).
! R1 LAN 192.168.10.0/24 on G0/0, R2 LAN 192.168.20.0/24, serial link 10.0.0.0/30.
enable
configure terminal
hostname R1
interface GigabitEthernet0/0
ip address 192.168.10.1 255.255.255.0
 no shutdown
! TODO 1: Configure Serial0/0/0 with 10.0.0.1/30, set the clock rate to 64000 (R1 is the
!   DCE end) and bring it up

exit
! TODO 2: Add a static route to the City campus LAN 192.168.20.0/24 through the next
hop
!   10.0.0.2

end
write memory
show ip route
```

Expected result: show ip route on R1 lists 192.168.20.0/24 as a static route (S) through 10.0.0.2, and a PC at the main campus can ping a PC at the city campus. Remove the clock rate on the DCE end and record what happens to the serial link.

Alternative Concept Analysis

Replace the five static routes with OSPF on both campuses. Count the configuration lines needed for 2 campuses and for 5 campuses in each approach, and compare the time to recover when the main link fails.

Evidence Expected

Correct next hops with the longest-prefix rule, a working Packet Tracer network with a floating static route, and before-and-after routing tables.

Solution:

1. Longest-prefix match

Destination	Best matching route	Next hop	Reason
10.1.2.200	10.1.2.128/25	192.0.2.13	Matches /8, /16, /24 and /25; /25 is longest
10.1.2.100	10.1.2.0/24	192.0.2.9	Address is outside 10.1.2.128/25
10.1.9.9	10.1.0.0/16	192.0.2.5	Matches /8 and /16; /16 is longer
10.7.7.7	10.0.0.0/8	192.0.2.1	Matches the /8 route only among specific routes
8.8.8.8	0.0.0.0/0	198.51.100.1	Uses the default route

2. R1 configuration (primary link)

Enable

configure terminal

hostname R1

interface GigabitEthernet0/0

ip address 192.168.10.1 255.255.255.0

no shutdown

exit

interface Serial0/0/0

ip address 10.0.0.1 255.255.255.252

clock rate 64000

no shutdown

exit

ip route 192.168.20.0 255.255.255.0 10.0.0.2

end

write memory

show ip route

Apply clock rate only on the DCE end of the serial cable. If the actual Packet Tracer router uses different interface names or the DCE end is on R2, adapt the interface name and clock-rate placement.

3. R2 reverse route

Enable

configure terminal

hostname R2

interface GigabitEthernet0/0

ip address 192.168.20.1 255.255.255.0

no shutdown

exit

interface Serial0/0/0

ip address 10.0.0.2 255.255.255.252

no shutdown

exit

ip route 192.168.10.0 255.255.255.0 10.0.0.1

end

write memory

Set the PCs to addresses in their LANs, with default gateways 192.168.10.1 and 192.168.20.1. Verify with ping in both directions and check `show ip route`.

4. Floating static backup link

Example backup subnet: 10.0.1.0/30, with R1 = 10.0.1.1 and R2 = 10.0.1.2 on a second serial link. Configure both ends and ensure the backup link is up.

! R1: primary route has default AD 1; backup route has AD 200

ip route 192.168.20.0 255.255.255.0 10.0.0.2

ip route 192.168.20.0 255.255.255.0 10.0.1.2 200

! R2: reverse routes

ip route 192.168.10.0 255.255.255.0 10.0.0.1

ip route 192.168.10.0 255.255.255.0 10.0.1.1 200

Test: ping between the two PCs, then shut down the primary serial interface at both ends (or disconnect the primary cable). Check `show ip route` and ping again. The backup route should be installed if its next hop remains reachable. Record actual results and screenshots.

5. Static routes vs OSPF: configuration-line comparison

Assumption for a fair count: N campus routers in a hub-and-spoke topology, one LAN per campus, and one OSPF network statement for each LAN and each router-side WAN interface.

Counts below include routing-protocol statements only; interface IP commands and optional OSPF settings are excluded.

Campuses	Static route statements (N × (N−1))	OSPF process statements	OSPF network statements	Total OSPF statements
2	2	2	4	6
5	20	5	13	18

Static routing is simple for two campuses, but the number of remote routes grows quickly. OSPF exchanges link-state information and recalculates routes after a failure. Actual recovery time depends on hello/dead timers, topology, device processing, and simulation behavior; measure it in Packet Tracer rather than inventing a number.

Q4 – Watch Distance-Vector Routing Count to Infinity

Bloom's Level: K5 – Evaluate

Problem Context – District Government Offices

Five routers connect government offices with the link costs A–B 2, A–C 7, B–C 3, B–D 8, C–D 1, D–E 4.

Your Challenge

- Build every router's distance table round by round until nothing changes, and state how many rounds it took
- Explain why A's best route to C is not the direct link
- Now take three routers in a line X–Y–Z (cost 1 each). The link Y–Z fails. Show, round by round, how X and Y keep raising their distance to Z (count to infinity), and how split horizon with poison reverse stops it
- Configure RIP in Packet Tracer and observe the convergence after a link failure

Complete the Code

Complete TODO 1 below, and run the program with Python 3.

```
"""Distance-vector routing (Bellman-Ford exchange) among five campus routers."""
INF = 99
names = "ABCDE"
cost = {("A", "B"): 2, ("A", "C"): 7, ("B", "C"): 3, ("B", "D"): 8,
        ("C", "D"): 1, ("D", "E"): 4}
link = {}
for (u, v), c in cost.items():
link[u, v] = link[v, u] = c

table = {u: {v: (0 if u == v else link.get((u, v), INF)) for v in names} for u in names}

rounds = 0
changed = True
while changed:
    changed = False
    rounds += 1
    old = {u: dict(table[u]) for u in names}       # vectors received this round
    for u in names:
        for n in names:
            if (u, n) not in link:
                continue
            for dest in names:
                # TODO 1: Bellman-Ford update: if going through neighbour n (link cost +
                #   n's old distance to dest) is cheaper than table[u][dest], update it
                #   and set changed = True
pass  # replace this line with your code
print(f"converged after {rounds} rounds")
for u in names:
print(u, " ".join(f"{table[u][v]:2d}" for v in names))
```

Expected output:

```
converged after 4 rounds
A  02  5  6 10
B  20  34  8
C  53  01  5
D  64  10  4
E 10  85  4  0
```

After convergence, set the cost of link C–D to 99 (failed) and rerun; record the new tables and rounds.

Alternative Concept Analysis

Run the same five-router network with link-state routing instead. After the C–D link fails, compare how many messages each method sends and how many rounds pass before every router has a correct route.

Evidence Expected

Correct round-by-round tables, a traced count-to-infinity example, the effect of poison reverse and RIP evidence from Packet Tracer.

Solution:

1. **Converged distance table**

   Use INF = 99. The shortest path from A to C is A–B–C with cost 2 + 3 = 5, cheaper than the direct A–C cost of 7.

Router / destination	A	B	C	D	E
A	0	2	5	6	10
B	2	0	3	4	8
C	5	3	0	1	5
D	6	4	1	0	4
E	10	8	5	4	0

2. **Completed Bellman-Ford distance-vector program**

```
"""Distance-vector routing (Bellman-Ford exchange) among five campus routers."""
INF = 99
names = "ABCDE"
cost = {
    ("A", "B"): 2, ("A", "C"): 7, ("B", "C"): 3,
    ("B", "D"): 8, ("C", "D"): 1, ("D", "E"): 4
}
link = {}
for (u, v), c in cost.items():
    link[u, v] = link[v, u] = c
table = {
    u: {v: (0 if u == v else link.get((u, v), INF)) for v in names}
    for u in names
}
rounds = 0
changed = True
```

```
while changed:
    changed = False
    rounds += 1
    old = {u: dict(table[u]) for u in names}
    for u in names:
        for n in names:
            if (u, n) not in link:
                continue
            for dest in names:
                new_cost = link[u, n] + old[n][dest]
                if new_cost < table[u][dest]:
                    table[u][dest] = new_cost
                    changed = True
print(f"converged after {rounds} rounds")
for u in names:
    print(u, " ".join(f"{table[u][v]:2d}" for v in names))
```

Output for this implementation:

```
Output

converged after 4 rounds
A  0  2  5  6 10
B  2  0  3  4  8
C  5  3  0  1  5
D  6  4  1  0  4
E 10  8  5  4  0
```

Round-by-round distance tables (column order A, B, C, D, E; 99 means unknown/unreachable in the initial vector):

Initial table

Router	**A**	**B**	**C**	**D**	**E**
A	0	2	7	99	99
B	2	0	3	8	99
C	7	3	0	1	99
D	99	8	1	0	4
E	99	99	99	4	0

After round 1

Router	**A**	**B**	**C**	**D**	**E**
A	0	2	5	8	99
B	2	0	3	4	12
C	5	3	0	1	5
D	8	4	1	0	4
E	99	12	5	4	0

After round 2

Router	A	B	C	D	E
A	0	2	5	6	12
B	2	0	3	4	8
C	5	3	0	1	5
D	6	4	1	0	4
E	12	8	5	4	0

After round 3

Router	A	B	C	D	E
A	0	2	5	6	10
B	2	0	3	4	8
C	5	3	0	1	5
D	6	4	1	0	4
E	10	8	5	4	0

After round 4 (no change)

Router	A	B	C	D	E
A	0	2	5	6	10
B	2	0	3	4	8
C	5	3	0	1	5
D	6	4	1	0	4
E	10	8	5	4	0

3.Link C–D fails (cost changed to 99)

Following the assignment instruction, replace the C–D cost with 99 and rerun the same program. The resulting shortest-path table is:

Router / destination	A	B	C	D	E
A	0	2	5	10	14
B	2	0	3	8	12
C	5	3	0	11	15
D	10	8	11	0	4
E	14	12	15	4	0

The route C to D now uses C–B–D with cost 3 + 8 = 11. This is the shortest remaining path. In a real failure simulation, remove/disable the link; setting its cost to 99 is the assignment's simplified unreachable-cost convention.

4. Count to infinity and poison reverse

For X–Y–Z with link costs X–Y = 1 and Y–Z = 1, before failure X reaches Z at cost 2 through Y, and Y reaches Z directly at cost 1.

Stage (illustrative asynchronous updates)	**X's advertised distance to Z**	**Y's advertised distance to Z**	**Explanation**
Before failure	2	1	X uses Y; Y uses direct Y–Z link.
Y–Z fails	2 (stale)	∞	Y loses its direct route; X has not yet learned of the failure.
Y hears stale X route	2	3 via X	Y mistakenly believes X has another route to Z.
X hears Y's new route	4 via Y	3	X raises its cost to 1 + 3.
Further exchanges	6, 8, …	5, 7, …	Distances rise until protocol infinity is reached.

Exact update values/order depend on update timing and the protocol's infinity limit. Split horizon with poison reverse stops this two-router loop: X advertises an infinite metric for Z back to Y because X learned Z through Y, so Y will not choose X as an alternate path to Z.

5. RIP Packet Tracer evidence

```
! Example on each router; replace network statements with the networks

! actually connected to that router.

router rip

 version 2

 no auto-summary

 network 10.0.0.0

 network 192.168.1.0
```

Build the five-router topology with the given link costs represented by the chosen interface/serial links. Configure RIP v2 and advertise the connected networks. Use `show ip route`, `show ip protocols`, and ping before/after shutting down a link. Capture actual convergence behavior. RIP hop count is its route metric; the assignment's weighted costs are for the Bellman-Ford exercise, not a direct RIP cost configuration.

6. Distance-vector vs link-state

Distance-vector routers exchange route-distance vectors with neighbours and may require multiple rounds for changes to propagate; loops/count-to-infinity are possible. Link-state routers flood link-state advertisements, build a common topology database, and run Dijkstra locally. Link-state typically converges faster and avoids the classic distance-vector count-to-infinity behavior, but it uses more CPU/memory and has flooding overhead. Exact message counts and recovery time must be captured from the chosen simulator and topology; they cannot be determined exactly from the assignment alone.

Q5 – Why Does OSPF Treat 1 Gbps and 10 Gbps the Same?

Bloom's Level: K5 – Evaluate

Problem Context – A State Internet Service Provider

An internet service provider connects cities with 10 Mbps, 100 Mbps, 1 Gbps and 10 Gbps links. After moving to OSPF, traffic still uses a 1 Gbps path instead of a parallel 10 Gbps path.

Your Challenge

- Calculate the OSPF cost of each link speed with the default reference bandwidth of 100 Mbps, and explain the problem
- Choose a new reference bandwidth, recalculate the costs, and run Dijkstra's algorithm by hand from one router on a 6-router network of your design
- Configure OSPF in Packet Tracer (with auto-cost reference-bandwidth) and show that the 10 Gbps path is now chosen
- Decide whether the provider should use one OSPF area or several, and justify the decision

Alternative Concept Analysis

The provider could use EIGRP (which uses bandwidth and delay in its metric) or IS-IS instead of OSPF. Compute the metric each would give the 1 Gbps and 10 Gbps paths, and decide which protocol avoids the problem without changing the reference bandwidth.

Evidence Expected

Correct cost calculations, a hand-worked shortest-path tree, Packet Tracer evidence and a justified area design.

Solution:

1. **Default reference-bandwidth calculation**

   OSPF interface cost is approximately reference bandwidth ÷ interface bandwidth, represented as an integer. With the default reference bandwidth of 100 Mbps, high-speed links can all round/truncate to cost 1.

Link speed	Calculation at 100 Mbps reference	OSPF cost
10 Mbps	100 / 10	10
100 Mbps	100 / 100	1
1 Gbps (1000 Mbps)	100 / 1000 = 0.1	1 (minimum/integer cost)
10 Gbps (10000 Mbps)	100 / 10000 = 0.01	1 (minimum/integer cost)

2. New reference bandwidth

Choose a consistent reference bandwidth of 100,000 Mbps (100 Gbps) on all OSPF routers.

Link speed	Calculation at 100,000 Mbps reference	Cost
10 Mbps	100000 / 10	10000
100 Mbps	100000 / 100	1000
1 Gbps	100000 / 1000	100
10 Gbps	100000 / 10000	10

router ospf 1

auto-cost reference-bandwidth 100000

Configure the reference bandwidth consistently on every router. If the Packet Tracer version does not accept that value or does not model the interface speed as expected, set explicit OSPF interface costs and verify the route with `show ip route` and `show ip ospf interface`.

3. Six-router design and Dijkstra calculation

Use these undirected links and the costs derived from the new reference bandwidth:

Link	Bandwidth	OSPF cost
R1–R2	1 Gbps	100
R1–R3	10 Gbps	10
R2–R4	10 Gbps	10
R3–R4	1 Gbps	100
R3–R5	100 Mbps	1000
R4–R6	10 Gbps	10
R5–R6	1 Gbps	100

Run Dijkstra from R1:

Destination	Selected shortest path	Total cost
R1	R1	0
R2	R1–R2	100
R3	R1–R3	10
R4	R1–R3–R4 or R1–R2–R4	110 (tie)
R6	R1–R3–R4–R6	120
R5	R1–R3–R5	1010

This example shows that a lower-cost high-speed route may be preferred; the exact selected path depends on the sum of configured link costs, not bandwidth alone.

4. One area vs multiple OSPF areas

For a six-router network, one area (Area 0) is usually simpler and appropriate if the topology is small and stable.

Multiple areas can reduce topology information and SPF scope for larger networks, support hierarchy, and improve summarisation, but require area-border routers and more design/configuration.

Recommendation: use one area for the six-router demonstration; consider multiple areas as the provider grows.

5. EIGRP or IS-IS alternative analysis

Classic EIGRP's default metric uses minimum path bandwidth and cumulative delay. Assuming equal delay for both compared paths, the bandwidth component is proportional to 10^7 / bandwidth in kbps, multiplied by 256.

Path interface bandwidth	**10^7 / bandwidth (kbps)**	**Bandwidth-only scaled term (×256)**
1 Gbps = 1,000,000 kbps	10	2560
10 Gbps = 10,000,000 kbps	1	256

These are bandwidth-only illustrative terms, not complete EIGRP metrics; a real calculation must include the configured cumulative delay and interface bandwidth values. With the same delay, the 10 Gbps path has the lower metric.

IS-IS uses configurable link metrics; it does not automatically derive a unique metric from bandwidth in every configuration. If the operator assigns metric 100 to a 1 Gbps link and 10 to a 10 Gbps link, IS-IS will prefer the 10 Gbps link where the total path cost is lower. Thus

EIGRP can incorporate bandwidth/delay by default, while IS-IS requires an appropriate metric policy. Neither protocol's exact result can be computed without topology and metric settings.

Q6 – IPv6, ICMP, Multicast and SDN in a Smart City

Bloom's Level: K5 – Evaluate

Problem Context – Modernising a Smart-City Network

A smart city will connect 2 million sensors, send each traffic-camera stream to several control rooms, and wants to change routing centrally.

Your Challenge

- Write 2001:0db8:0000:0000:0000:ff00:0042:8329 in its shortest form, and build the EUI-64 interface identifier for a sensor with MAC address 00:1A:2B:3C:4D:5E
- Explain, using the TTL field, how traceroute uses ICMP Time Exceeded messages to discover each router
- Find the multicast MAC address for the group 239.1.2.3, and explain why one camera stream reaches several control rooms without being copied by the camera
- An OpenFlow switch has the rules (priority 200: dst 10.5.0.0/16 → port 3), (priority 100: TCP port 80 → controller), (priority 0: drop). Decide what happens to an HTTP packet to 10.5.1.1 and to an SSH packet to 10.6.1.1, and judge whether SDN is worth adopting for the city

Alternative Concept Analysis

Compare moving straight to IPv6 with running dual-stack or tunnelling IPv6 over IPv4 for the city. Calculate the extra header bytes of a 6in4 tunnel on a 100-byte sensor packet, and decide which approach the city should start with.

Evidence Expected

Correct IPv6, EUI-64 and multicast calculations, a correct traceroute explanation and a reasoned SDN decision.

Solution:

1. IPv6 compression

Full address: 2001:0db8:0000:0000:0000:ff00:0042:8329

Shortest form: 2001:db8::ff00:42:8329

2. EUI-64 from MAC address

MAC: 00:1A:2B:3C:4D:5E. Insert FF:FE between the first and last three bytes, then invert the universal/local bit in the first byte (00 becomes 02).

Modified EUI-64: 02:1A:2B:FF:FE:3C:4D:5E

Interface identifier: 021A:2BFF:FE3C:4D5E

3. Traceroute and ICMP

Traceroute sends a probe with TTL/Hop Limit 1.

The first router decrements TTL to zero, discards the probe, and usually sends ICMP Time Exceeded.

Traceroute increases TTL to 2, then 3, and so on; the responding router identifies each hop.

IPv4 uses TTL and ICMP Time Exceeded; IPv6 uses Hop Limit and ICMPv6 Time Exceeded. Filtering or lost responses can cause a hop to appear as a timeout.

4. IPv4 multicast MAC address

IPv4 multicast group: 239.1.2.3. The Ethernet mapping uses 01:00:5E plus the low 23 bits of the IPv4 multicast address.

Multicast MAC: 01:00:5E:01:02:03

The camera sends one multicast stream to the group address. Multicast-capable routers/switches replicate it at the network where necessary for interested receivers, rather than requiring the camera to originate a separate unicast stream for every control room. IGMP snooping and multicast routing should be configured where appropriate.

5. OpenFlow rules

Packet	Matching rules	Result
HTTP to 10.5.1.1 (TCP destination port 80)	Priority 200 destination 10.5.0.0/16 and priority 100 TCP/80	Priority 200 wins; forward to port 3
SSH to 10.6.1.1 (TCP destination port 22)	No destination /16 match and no TCP/80 match; default priority 0	Drop

SDN is worth considering for the smart city because it centralises policy and can simplify traffic engineering, monitoring, and security changes. Trade-offs include controller availability, controller security, vendor interoperability, and operational complexity. Use a resilient controller design and secure the control channel.

6. IPv6 migration and 6in4 overhead

Approach	Benefit	Limitation
IPv6-only	Clean long-term design and large address space	IPv4-only services need translation/proxies or cannot communicate directly
Dual-stack	IPv4 and IPv6 coexist during migration	Both protocols must be operated, secured, and monitored
6in4 tunnelling	Carries IPv6 through existing IPv4 infrastructure	Adds encapsulation overhead and tunnel-management complexity

A basic 6in4 tunnel adds a 20-byte outer IPv4 header (assuming no IPv4 options). A 100-byte inner packet becomes 120 bytes on the outer IPv4 path: 20 extra bytes, a 20% increase relative to the original 100-byte packet.

Recommendation: start with dual-stack where the city's devices and services support it, and use tunnels only where needed to cross IPv4-only infrastructure. For constrained sensors, assess MTU and packet overhead carefully.

DAY-4 DESIGN CHALLENGE

Q7 – Find the Weak Point, Redesign and Validate

Bloom's Level: K5 – Evaluate

Review the evidence you generated in Q1–Q6. Select **ONE** network that shows a significant limitation as it grows or as the requirement changes.

You are **not** permitted simply to add more equipment and claim the network is better. You must understand the limitation first and then develop an appropriate redesign.

A. Identify the Limitation

Using your own evidence, show where the design fails or slows down and which part of the addressing or routing design causes it.

Evidence Expected

A precise analysis supported by evidence.

B. Define the Design Objective

State a measurable objective (for example: convergence time, addresses wasted, routing-table size).

Evidence Expected

A clear, measurable objective.

C. Design the Revised Network

Explain what you have changed and any new trade-off. Provide the topology diagram and address table.

Evidence Expected

A coherent redesign directly addressing the limitation.

D. Predict Before Implementing

Predict the improvement and the measure in which you expect it.

Evidence Expected

A clear, testable prediction.

E. Implement and Validate

Compare **Original → Redesigned** in Packet Tracer using the same tests.

Evidence Expected

Comparable evidence for both versions.

F. Defend the Final Design

Judge whether the objective was achieved, the new costs, and where the original may still be preferable.

Evidence Expected

A critical, evidence-based conclusion.

Solution:

Selected limitation from Q2: the original 192.168.10.0/24 VLSM plan supports the current Labs, Staff, Library and WAN needs, but cannot fit the additional 60-host hostel subnet. The five required subnet blocks need 292 addresses, more than a /24 can provide.

A. Identify the limitation

The original pool contains 256 total addresses. Labs /25 uses 128, Staff /26 uses 64, Library /27 uses 32, and WAN /30 uses 4. Adding Hostel /26 requires another 64 addresses: 128 + 64 + 32 + 4 + 64 = 292. Therefore, no renumbering inside the same /24 can satisfy all five subnet-size requirements.

B. Measurable design objective

Objective: design a non-overlapping addressing plan that supports Labs (100 hosts), Staff (50), Hostel (60), Library (25), and a two-host WAN link; all networks must have enough usable host addresses and successful end-to-end ping tests.

C. Revised network design

Expand the contiguous pool to 192.168.10.0/23 (512 total addresses, spanning 192.168.10.0–192.168.11.255) and allocate the largest blocks first.

Network	Network/prefix	Mask	First host	Last host	Broadcast	Gateway example
Labs (100)	192.168.10.0/25	255.255.255.128	192.168.10.1	192.168.10.126	192.168.10.127	192.168.10.1
Staff (50)	192.168.10.128/26	255.255.255.192	192.168.10.129	192.168.10.190	192.168.10.191	192.168.10.129
Hostel (60)	192.168.10.192/26	255.255.255.192	192.168.10.193	192.168.10.254	192.168.10.255	192.168.10.193
Library (25)	192.168.11.0/27	255.255.255.224	192.168.11.1	192.168.11.30	192.168.11.31	192.168.11.1
WAN (2)	192.168.11.32/30	255.255.255.252	192.168.11.33	192.168.11.34	192.168.11.35	Use .33 and .34 on router ends

Topology sketch:

Labs PCs --- Switch --- R1 --- Staff PCs

|

+--- Library PCs

|

+--- Hostel PCs

|

WAN /30

|

R2 --- Remote test LAN

Implementation note: use a router with enough Ethernet interfaces, or implement the departmental LANs as VLANs with router-on-a-stick/L3 switching. Configure each gateway address above. Use a separate remote test LAN behind R2 if testing routed WAN connectivity.

D. Prediction before implementation

Prediction: the /23 VLSM plan will accommodate all five required networks without overlap, and each network will have at least the required number of usable addresses. The original /24 plan will fail the hostel capacity requirement.

E. Implement and validate

In Packet Tracer, configure the original design and the redesigned /23 design with the same host counts/test pattern. For each version:

Assign addresses and correct masks/gateways to representative PCs in each subnet.

Use `show ip interface brief` to check interface status and `show ip route` to check connected/static routes.

Ping between representative PCs in the same subnet and between routed subnets.

Record the address capacity result and the exact ping outcomes; save both .pkt files and screenshots.

Measure	**Original /24 design**	**Redesigned /23 VLSM design**
Current Labs/Staff/Library/WAN requirements	Fits using VLSM	Fits
Additional 60-host Hostel	Does not fit all subnet blocks inside /24	Fits in 192.168.10.192/26
Subnet overlap	None in original four-subnet plan, but no room for hostel	None if configured as table
Host communication	Test with same pings	Test with same pings

Do not write that the pings succeeded until you run them. Add screenshots of actual output to the final report.

F. Defend the final design

The /23 VLSM design achieves the stated address-capacity objective by providing an aligned /26 for the hostel while preserving suitable sizes for the other departments. Its trade-off is consuming a larger address pool (512 total addresses) and requiring coordinated renumbering/address planning. The original /24 remains preferable if the hostel is not added and address conservation is the main priority.

Conclusion: the revised /23 VLSM plan is scalable for the stated growth requirement, provided the expanded address allocation is approved and the Packet Tracer tests confirm connectivity.